Week-12

Design principle

Design principles in Java are best practices and guidelines for writing **clean, maintainable, and scalable object-oriented code**. The most prominent set of these principles are the **SOLID** principles, along with several general OOP guidelines.

SOLID Principles

Coined by Robert C. Martin, these five principles promote robust software design:

- **SRP (Single Responsibility):** Classes should have one focused job, simplifying maintenance.
- **OCP (Open/Closed):** Code should be open for extension but closed for modification, often achieved via interfaces.
- **LSP (Liskov Substitution):** Child classes must be able to replace parent classes without breaking functionality.
- **ISP (Interface Segregation):** Clients should not rely on interfaces they don't use; prefer small, specific interfaces.
- **DIP (Dependency Inversion):** Depend on abstractions (interfaces), not concrete implementations

Interface segregation principle [ISP]

The **Interface Segregation Principle (ISP)** in Java is a design guideline stating that **clients should not be forced to depend on methods they do not use.** It advocates for creating smaller, specific, and highly cohesive interfaces rather than large, general-purpose ("fat") ones.

The core idea is to break down interfaces into role-specific ones, ensuring that implementing classes only need to provide meaningful implementations for methods relevant to their functionality.

Real-Life Example

Think of a **TV remote** 📺

❌ Bad:

- One remote with **100 buttons**
- You only use 10 → confusing

✅ Good:

- Separate remotes:
  - TV remote
  - AC remote
- You only use what you need

ISP Violation (Bad Design)

```
          +----------------------+
          |     Worker           |
          +----------------------+
          | work()               |
          | eat()                |
          +----------------------+
             /      \
            /        \
    +----------------+  +----------------+
    |    Robot       |  |    Human       |
    +----------------+  +----------------+
    | work()         |  | work()         |
    | eat() ❌       |  | eat()          |
    +----------------+  +----------------+
```

Problem:

- **Robot does NOT eat**, but still forced to implement eat()
- This **violates ISP** (forcing unnecessary methods)

Example :

```
interface Worker {
   void work();
   void eat();
}
class Robot implements Worker {
   public void work() {
      System.out.println("Robot working");
   }
```

```
    public void eat() {
        // not needed
    }
}
```

ISP Followed (Good Design)

```
      +------------------+      +------------------+
      |   Workable       |      |   Eatable        |
      +------------------+      +------------------+
      | work()           |      | eat()            |
      +------------------+      +------------------+
            |                   |
      ------------------      ------------------
      |              |      |              |
+---------------+  +---------------+   +---------------+
|   Robot       |  |   Human       |   |   Human       |
+---------------+  +---------------+   +---------------+
| work()        |  | work()        |   | eat()         |
+---------------+  +---------------+   +---------------+
```

✅ **Benefit:**

- Robot only implements work()
- Human implements both work() and eat()
- No unnecessary methods → **clean design**

Example:

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}
```

```
class Robot implements Workable {
    public void work() {
        System.out.println("Robot working");
    }
}

class Human implements Workable, Eatable {
    public void work() {
        System.out.println("Human working");
    }

    public void eat() {
        System.out.println("Human eating");
    }
}
```

Benefits:

1. Avoids Unnecessary Methods

- Classes **don't have to implement methods they don't need**
- Keeps code clean and meaningful

2. Improves Code Readability

- Small and focused interfaces are **easy to understand**
- Developers quickly know what each interface does

3. Increases Flexibility

- You can **combine interfaces as needed**

4. Easier Maintenance

- Changes in one interface **won't affect unrelated classes**
- Reduces risk of breaking code

5. Promotes Loose Coupling

- Classes depend only on what they actually use
- Reduces dependency between components

6. Better Testability

- Small interfaces are **easy to test individually**
- You can mock only required behavior in testing

7. Supports Reusability

- Small interfaces can be reused in different classes/projects

8. Follows SOLID Design Principles

- ISP is one of the **SOLID principles**
- Helps build **scalable and professional applications**

Enums

An **enum** (enumeration) in Java is a special **type-safe** data type used to define a fixed set of constants. Enums are useful when all possible values are known at compile time, such as days of the week, directions (North, South, East, West), or the states of a process (running, waiting, dead).

Key Characteristics

- **Type-Safety:** Enums prevent the use of invalid values by restricting a variable to one of the predefined constants, which helps eliminate bugs.
- **Constants:** The values defined within an enum are implicitly public, static, and final. By convention, they are written in all uppercase letters.
- **Object-Oriented:** Unlike enumerations in some other languages, Java enums are full-fledged classes that implicitly extend the abstract class java.lang.Enum.
- **Functionality:** Enums can have constructors, methods, and fields, and can also implement interfaces.

Syntax:

```
Enum enumName {

CONSTANT1, CONSTANT2, CONSTANT3, CONSTANT4,…

}
```

Example:

1) Enum Direction {

NORTH, SOUTH, EAST, WEST

}

2)enum Level {

  LOW,

  MEDIUM,

  HIGH

}

You can access enum constants with the **dot** syntax:

Level myVar = Level.MEDIUM;

Enum inside a Class

You can also have an enum inside a class:

Example

```
public class Main {
  enum Level {
    LOW,
    MEDIUM,
    HIGH
  }

  public static void main(String[] args) {
    Level myVar = Level.MEDIUM;
    System.out.println(myVar);
  }
}
```

The output will be:

MEDIUM

Enum in a Switch Statement

Enums are often used in switch statements to check for corresponding values:

Example

```
enum Level {
  LOW,
  MEDIUM,
  HIGH
}

public class Main {
  public static void main(String[] args) {
    Level myVar = Level.MEDIUM;

    switch(myVar) {
      case LOW:
        System.out.println("Low level");
        break;
      case MEDIUM:
         System.out.println("Medium level");
        break;
      case HIGH:
        System.out.println("High level");
        break;
    }
  }
}
```

The output will be:

Medium level

<u>Loop Through an Enum</u>

The enum type has a values() method, which returns an array of all enum constants. This method is useful when you want to loop through the constants of an enum:

Example

```
for (Level myVar : Level.values()) {
  System.out.println(myVar);
}
```

The output will be:

```
LOW
MEDIUM
HIGH
```

Difference between Enums and Classes

An enum can, just like a class, have attributes and methods. The only difference is that enum constants are public, static and final (unchangeable - cannot be overridden).

An enum cannot be used to create objects, and it cannot extend other classes (but it can implement interfaces).

Why And When To Use Enums?

Use enums when you have values that you know aren't going to change, like month days, days, colors, deck of cards, etc.

Annotations in Java

Annotations in Java are a form of metadata that provide additional information about the program. They do not change the action of a compiled program but can be used by the compiler or runtime for processing.

Key Points

- Annotations start with '@'.
- Annotations do not change the action of a compiled program.
- Annotations help to associate metadata (information) to the program elements i.e. instance variables, constructors, methods, classes, etc.
- Annotations are not pure comments as they can change the way a program is treated by the compiler. See below code for example.
- Annotations basically are used to provide additional information, so could be an alternative to XML and Java marker interfaces.

Categories of Annotations

There are broadly 5 categories of annotations as listed:

1. Marker Annotations
2. Single value Annotations
3. Full Annotations
4. Type Annotations
5. Repeating Annotations

Let us discuss and we will be appending code wherever required if so.

1. Marker Annotations

- Do not have any elements or values (just presence is enough).
- Used to signal the compiler or tools with metadata.

Example

@TestAnnotation()

2. Single value Annotations

- Contain only one element (can use shorthand notation).
- When specifying value, you don't need to write the element name.

Example

@TestAnnotation("testing");

3. Full Annotations

- Contain multiple elements as key-value pairs.
- All values must be provided unless defaults are defined.

Example

@TestAnnotation(owner="Rahul", value="Class Geeks")

4. Type Annotations

- Introduced in Java 8 for annotating types (variables, generics, return types).
- Useful for stronger type checking and frameworks.
- Declared with @Target(ElementType.TYPE_USE)

5. Repeating Annotations

- Allow the same annotation to be applied multiple times on a single element.
- Declared using @Repeatable with a container annotation.
- **Example:** @Schedule(day="Monday") @Schedule(day="Tuesday").

Built in java Annotations

1. @Deprecated

- It is a marker annotation. It indicates that a declaration is obsolete and has been replaced by a newer form.
- The Javadoc @deprecated tag should be used when an element has been deprecated.
- @deprecated tag is for documentation and @Deprecated annotation is for runtime reflection.
- @deprecated tag has higher priority than @Deprecated annotation when both are together used.

2. @Override

- It is a marker annotation that can be used only on methods.
- Ensures that a method actually overrides a method from its superclass.
- Helps catch errors when method signatures don't match.

3. @SuppressWarnings

- Suppresses specific compiler warnings (e.g., unchecked, deprecation).
- Useful to make code cleaner without unnecessary warning messages.

4. @Documented

- Marks annotations to be included in generated Javadoc.
- Improves API documentation clarity.

5. @Target

- Specifies where the annotation can be applied (class, method, field, etc.).
- Prevents misuse of annotations in wrong places.

6. @Inherited

- Allows a subclass to inherit an annotation from its parent class.

- Useful in frameworks where annotations define behavior across hierarchies.

User-defined (Custom) Annotation

User-defined annotations can be used to annotate program elements, i.e. variables, constructors, methods, etc. These annotations can be applied just before the declaration of an element (constructor, method, classes, etc).

Syntax

[Access Specifier] @interface<AnnotationName>

{
DataType <Method Name>() [default value];
}

Do keep these certain points as rules for custom annotations before implementing user-defined annotations.

- AnnotationName is an interface.
- The parameter should not be associated with method declarations and throws clause should not be used with method declaration.
- Parameters will not have a null value but can have a default value.
- default value is optional.
- The return type of method should be either primitive, enum, string, class name or array of primitive, enum, string or class name type.